CANDY
DIALOGUE ENGINE

Effects

1.1.0

Effect commands control various visual effects.

§Effect

The §Effect command is used for playing an animation through an AnimationPlayer node.

Node

The **Animation Player** field must be provided a path key to an AnimationPlayer. You can do this in several ways:

Path Keys

```
#^ Media Player paths:
#TODO: Edit paths and add media player path keys as needed.
var media_players_locations = {
        "Voice": "VoicePlayer",
        "PEffect": "Portrait/PortraitEffectPlayer",
        "Effect": "EffectPlayer",
        "Image": "ImagePlayer",
        "Splash": "SplashPlayer",
        "Video": "VideoPlayer",
        "Sound": "SoundPlayer",
        "Ambient": "AmbientPlayer",
        "Music": "MusicPlayer",
        "Speech": "SpeechPlayer",
}
```

The **media_player_locations** dictionary in **Candy_Engine.gd** (under the "Paths" region) can be used to store paths to various media player nodes that commands might use.

You can add any AnimationPlayer to the dictionary, then write the name of its key in the command's Node field.

Note that Candy DE will use the Candy_UI as the root node when looking for these players. If your players are not located under Candy_UI, write "/root/" at the start of their paths:

"MyPlayer": "/root/My_Scene/My_Player"

This is a simple, clean and reliable method of providing media players to commands that use them. It makes writing quicker, with less to remember.

Direct Path

If the AnimationPlayer you want to use for the §Effect command isn't featured in the **media_players_locations dictionary**, you can write a path in the Node field, similarly to how you would do it in GDScript:

$Candy_UI/AnimationPlayer

Note that in this case, '$' is actually the **node_symbol** (see the **Variables** guide). If you've customized it, you must use the actual symbol you chose.

Note also that when a path is provided in this way, Candy DE will look for it from /root/, not from Candy_UI.

Variables

Finally, you can also provide a variable that contains a path prefixed with the node_symbol or a reference to the AnimationPlayer node:

var which_player = "$Candy_UI/AnimationPlayer"

var which_player = get_node("Candy_UI/AnimationPlayer")

→ In either case, in the Node field, write the animation_player variable, for example:

£globals.which _player

$path.which _player

Animation

The **Animation** field requires the name of the animation you want to play. The AnimationPlayer must have an animation of the same name.

You can type a direct name, or use a variable that contains the name in the form of a string.

Loop

The **Loop** field indicates the number of times the animation should play before stopping. A value of 0 causes infinite looping. Set to 1 to play the animation only once.

Time & Wait

You can pause dialogue processing while the media is playing:

The **Wait Time** field represents the amount of time to pause dialogue for and is provided in the HH:MM:SS.00 format (Hours:Minutes:Seconds.decimals) which can be written multiple ways:

1:20:34 → 1 hour 20 minutes 34 seconds

03:44 → 3 minutes 44 seconds

140:05 → 140 minutes 5 seconds

0:130:90.3 → 0 hours, 130 minutes, 90.3 seconds

2.5 → 2.5 seconds

Whatever syntax you choose, Candy DE will calculate correctly as long as you use colon (:) to separate hours, minutes and seconds, and period (.) to separate decimals.

The **Wait Loops** field represents the number of times to let the animation play before resuming dialogue. 0 means dialogue won't wait. This is cumulative with **Wait Time**.

Multiple Effects

If you want to play multiple effects simultaneously, you'll need to use multiple successive §Effect commands, with **Wait Time** and **Wait Loops** set to 0: the engine will make them play at the same time.

Each §Effect command will also need to use a different AnimationPlayer, since an AnimationPlayer can only play one animation at a time. This isn't a problem: add more AnimationPlayer nodes to your dialogue UI, and add their paths in the media_players_locations dictionary. AnimationPlayers are lightweight and shouldn't impact performance.

If you want dialogue processing to pause until the animations finish, you can enable set **Wait Time** and **Wait Loops** on the last §Effect command. Make sure that last command is the one that triggers the longest animation, otherwise dialogue will resume while some animations are still playing.

Built-In Effects

Candy DE comes with a few pre-made animations for the §Effect commands, stored in the EffectPlayer node of the Candy_UI. These animations work on the ScreenEffects node, which is a simple ColorRect.

These effects are just simple fades and flashes, which many games are likely to use. You can of course modify, rename or delete them, or add your own.

§Effect_Wait

The §Effect_Wait command is used to pause dialogue processing and wait for an animation to complete. This can be used when the §Effect command that started an animation wasn't configured to pause dialogue processing (for example, in case you want to pause dialogue mid-way through the animation).

The **Animation Player**, **Wait Time** and **Wait Loops** fields are configured the same way as for the §Effect command: **Wait Time** is the amount of time to wait, and **Wait Loops** is the number of loops to wait on.

Both Wait Time and Wait Loops are calculated from the start of the animation. In other words, if you set **Wait Time** to '2:00', dialogue processing will pause until the animation has played for 2 minutes from the moment it began playing, not from the moment the §Effect_Wait command was processed. Likewise, a **Wait Loops** value of '3' will pause dialogue until the animation has completed 3 loops from the moment it began playing.

If the provided time and/or loops have already elapsed when the §Effect_Wait command is processed, dialogue won't pause.

As a reminder, **Wait Time** and **Wait Loops** are cumulative: Wait Time = '2:00' and Wait Loops = '3' means that dialogue processing should pause until the animation has played for 3 loops + 2 minutes.

§Effect_Stop

The §Effect_Stop command is used to stop an animation currently playing in an AnimationPlayer.

It only requires that you indicate which AnimationPlayer it should stop, in the **Animation Player** field. This can be provided in the same ways as for the §Effect command (see above).

Note that §Effect_Stop command will work on any AnimationPlayer: even those that weren't triggered by the §Effect command (i.e. animations started by your own code can be stopped).

§Wait

The §Wait command pauses dialogue processing for a set amount of time. It only requires that you provide the wait time, in seconds, in the **Wait Time** field. Decimal numbers (floats) are compatible.

The §Wait command is used for pacing dialogue: it can create pause line processing while animations or media loops, it can create pauses in conversations for narrative effect, or it can simply delay multiple successive commands if they shouldn't be performed at the same time.

§Hide

The §Hide command is used to hide dialogue display elements (e.g. the dialogue box, subtitles...) and portraits when video or image media are played.

When you play media with the §Video or §Image commands (see the Media Control guide), you have the option of showing UI elements such as dialogue and portrait displays *above* the media. This is done automatically by setting the Z index to a value higher than that of the media player being used.

§Hide simply resets the Z index of dialogue and portrait displays to their default, below media players, therefore hiding these UI Elements.

Show/Hide Text field: set to 1 to reset the Z index of the dialogue box, subtitles, chat box or speech bubbles. Leave empty or set to 0 to ignore these UI elements.

Show/Hide Portrait field: set to 1 to reset the Z index of the portrait box. Leave empty or set to 0 to ignore.

§Clear

The §Clear command is a more elaborate version of the §Hide command. It not only hides UI elements, it also removes their contents, such as images or text.

Unlike the §Hide command, which only affects dialogue and portrait displays, §Clear can also affect backgrounds and busts. Moreover, it lets you choose precisely which UI elements you want to clear.

The §Clear command is typically used to visually mark transitions in dialogues, such as time skips, conversation switching, or scene changes: §Clear will make dialogue and portrait displays momentarily disappear, before re-appearing blank when an actor speaks once more, preventing an unwanted impression of immediate continuity.